

Outline

- Specification of Leader Election
- YAIR
- Leader election in asynchronous rings:
 - An $O(n^2)$ algorithm
 - An $O(n \log(n))$ algorithm
- The revenge of the lower bound!
- Leader election in synchronous rings
 - Breaking the $\Omega(n \log(n))$ barrier

The LCR Algorithm

LeLann (1977), Chang and Roberts (1979)

- unidirectional
- asynchronous
- non anonymous: every process has *uid*
- uniform (does not depend on *n*)

```

1: upon receiving no message
2:   send  $uid_i$  to left (clockwise)

3: upon receiving  $m$  from right
4:   case
5:      $m.uid > uid_i$  :
6:       send  $m$  to left
7:      $m.uid < uid_i$  :
8:       discard  $m$ 
9:      $m.uid = uid_i$  :
10:      leader :=  $i$ 
11:      send <terminate,  $i$ > to left
12:      terminate
13:   endcase

13: upon receiving <terminate,  $i$ >
14:   from right
15:   leader :=  $i$ 
16:   send <terminate,  $i$ > to left
17:   terminate
  
```

Correctness

- messages from process with highest ID are never discarded
- therefore the correct leader is elected
- no other processor ID can traverse the entire ring
- therefore no one else is elected

Complexity

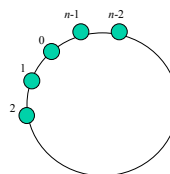
Message complexity:

Time complexity:

$O(n^2)$

$O(n)$

This bound is tight...



Can we do better?

The HS algorithm

Hirschberg and Sinclair (1980)

- Ring is bidirectional
- Each process p_i operates in *phases*
- In each phase r , p_i sends out "tokens" containing uid_i in both directions
- Tokens are intended to travel distance 2^r and return to p_i

- However, tokens may not make it back



- Token continues outbound only if greater than tokens on path
- Otherwise discarded
- All processes always forward tokens moving inbound

If p_i receives its own token while it is going outbound, p_i is the leader

The Protocol

```

0: Init: asleep := true
   phase := 0
1: upon receiving no message
2: if asleep then
3:   asleep := false
4:   send <probe, uid_i, 1, 0> to L and R

5: upon receiving <probe, uid_j, r, d>
   from L (resp. R)
6: if uid_j = uid_i then
7:   leader := p_i
8:   terminate
9: if (uid_j > uid_i and d < 2^r) then
10:  send <probe, uid_i, r, d+1> to R
    (resp. L)
11: if (uid_j > uid_i and d ≥ 2^r) then
12:  send <reply, uid_i, r> to L
    (resp. R)

13: upon receiving <reply, uid_j, r>
   from L (resp. R)
14: if uid_j ≠ uid_i then
15:  send <reply, uid_i, r> to R (resp. L)
16: else
17:  if already received <reply, uid_i, r>
   from R (resp. L) then
18:    send <probe, uid_i, r+1, 0> to L and R
  
```

Correctness

Same as LCR:

- messages from process with highest ID are never discarded
- therefore the correct leader is elected
- no other processor ID can traverse the entire ring
- therefore no one else is elected

Communication Complexity

- Every processor sends a token in phase 0
 $4n$ messages
- For phase $r > 0$,
 - the only processors to send a tokens are those who "won" in phase $r-1$
 - There is a winner for every $\lceil \frac{n}{2^{r-1}+1} \rceil$ processors (at most)
 - Winners in phase $r > 0$ $\lceil \frac{n}{2^{r-1}+1} \rceil$
 - Tokens travel distance 2^r
 - Total number of messages sent in phase r is bounded by $4 \cdot \lceil \frac{n}{2^{r-1}+1} \rceil \cdot 8n$
- Total number of phases $1 + \lceil \log n \rceil$
- No. of messages bound by $8n(1 + \lceil \log n \rceil)$ which is $O(n \log n)$

Time Complexity

- Time for each phase l $2 \cdot 2^r = 2^{r+1}$
 - Final phase takes n (tokens only traveling outbound)
 - Next to last phase is $r = \lfloor \log n \rfloor - 1$
 - Total time complexity excluding last phase $2 \cdot 2^{\lfloor \log n \rfloor}$
- Time complexity is at most $3n$ to $5n$

The revenge of the lower bound

So far we have seen:

- a simple $O(n^2)$ algorithm
- a more clever $O(n \log n)$ algorithm
- focus on message complexity

Facts:

- $\Omega(n \log n)$ lower bound in asynchronous networks
- $\Omega(n \log n)$ lower bound in synchronous networks when using only comparisons

Outline

- Specification of Leader Election
- YAIR
- Leader election in asynchronous rings:
 - An $O(n^2)$ algorithm
 - An $O(n \log(n))$ algorithm
- The revenge of the lower bound!
- Leader election in synchronous rings
 - Breaking the $\Omega(n \log(n))$ barrier
- The rise and fall of randomization

Leader Election with fewer than $O(n \log n)$ messages

- Synchronous rings
- UID are positive integers
- Can be manipulated using arbitrary arithmetic operations

TimeSlice

- n is known to all processors
- unidirectional communication
- $O(n)$ messages

VariableSpeeds

- n is *not* known to all processors
- unidirectional communication
- $O(n)$ messages

What about Time complexity?

What is special about synchronous rings?

- Can convey information by *not* sending a message

“when your phone doesn’t ring, it’s me”

TimeSlice

Runs in phases

- each phase consists of n rounds
- in phase $i \geq 0$
 - if no one elected yet
 - processor with id i
 - declares itself the leader
 - sends token with its UID around

Message complexity: n

Time complexity: $n \cdot \text{UID}_{\min}$

VariableSpeeds

- Each process p_i initiates a token
- Different tokens travel at different speeds:
 - for token carrying UID_i , 1 message every 2^{UID_i} rounds
 - (each process waits 2^{UID_i} rounds after receiving the token before sending it out)
- Each process keeps track of smallest UID seen
- Discard token with UID greater than smallest UID

Complexity Analysis

- By the time $\text{UID}_{\min}$ goes around the ring, the second smallest UID has gone only at most half way, third smallest at most a fourth of the way, etc.
- Forwarding the token carrying $\text{UID}_{\min}$ has caused more messages than all the other tokens combined
- Message complexity bound by $2n$
- Time Complexity $n \cdot 2^{\text{UID}_{\min}}$